# Genetic Algorithms for Test Suite Generation

## EvoSuite

# Announcements

- HW5 Due Monday November 2nd

# Genetic Algorithms

- Search technique used to find true or approximate solutions to search or optimization problems

- Use techniques inspired by evolutionary biology:
  - Selection
  - Mutation
  - Crossover

- Motivating Example: Imagine you're trying to create a **meal plan** that meets daily nutritional requirements while **minimizing cost**. This is an **optimization problem** where we search for the best combination of foods.

# Genetic Algorithms

## Selection (Survival of the Fittest)

- **Biology**: In nature, organisms that are better adapted to their environment are more likely to survive and reproduce. This is known as **natural selection**.

- **Optimization**: In evolutionary algorithms, the best-performing candidate solutions (individuals) are selected from a population based on their "fitness" (how well they solve the problem). These individuals are more likely to contribute to the next generation.

# Genetic Algorithms

## Crossover (Genetic Recombination)

- **Biology**: During reproduction, genetic material from two parents combines to create offspring with traits inherited from both, leading to increased genetic diversity.

- **Optimization**: In evolutionary algorithms, crossover combines parts of two parent solutions to create a new solution (offspring) that may inherit beneficial traits from both parents, improving the search process.

- **Meal plan example - what would crossover look like? Breakfast from M1 lunch from M2**

# Genetic Algorithms

**Mutation (Genetic Variation)**

- **Biology**: Mutations are random changes in an organism's DNA that introduce genetic diversity. While some mutations may be harmful, others can offer advantages that improve survival.

- **Optimization**: In evolutionary algorithms, mutation introduces small random changes to candidate solutions to maintain diversity and prevent the algorithm from getting stuck in local optima.

- **Meal plan example - what would mutation look like? Maybe swap out breakfast**

# Genetic Algorithms

- Start with a population of randomly generated individuals
- Works in multiple “generations”
  - Select individuals for breeding based on “fitness”
  - Mutation + crossover creates individuals for the next generation
  - Used in the next iteration

# Genetic Algorithm

**Termination**: The algorithm terminates when either:

- A max number of iterations have been reached or
- Timeout or
- A satisfactory fitness level has been achieved

**NO CONVERGENCE GUARANTEES**

# Vocabulary

- **Individual** - Any possible solution **(also called chromosome)**
- **Population** - Group of all individuals
- **Fitness** – Target function that we are optimizing (each individual has a fitness)

# A Generic Evolutionary Algorithm

- Also link the reading

**Algorithm 1** Evolutionary Algorithm

```
1: Initialize population P with random solutions
2: Evaluate fitness of each individual in P
3: while search budget not exhausted do
4:     Select individuals from P based on fitness
5:     Apply crossover to produce new offspring
6:     Apply mutation to offspring
7:     Evaluate fitness of offspring
8:     Add offspring to the population P
9: end while
```

# Warmup: Meal Plan GA

1. What is a chromosome?
   a. Meal plan with one option for breakfast and one for lunch
2. How are individuals selected?
   a. Probabilistically based on their fitness function
3. What happens during crossover?
   a. The new individual has breakfast from one plan and lunch from another
4. What happens during mutation?
   a. Breakfast is swapped out
5. Fitness function?
   a. F(meal)= nutrition− (0.5×cost)
   b. F(mealplan) = F(breakfast) + F(lunch)

# Warmup: Which Plan has the highest fitness?

| Plan | Breakfast | Lunch | Nutrition | Cost |
|---|---|---|---|---|
| **A** | Oatmeal | Salad | 16 | 17 |
| **B** | Eggs | Granola bar | 13 | 10 |
| **C** | Cereal | Wendy's | 3 | 7 |
| **D** | Oatmeal | Granola bar | 9 | 5 |

1. Using F = nutrition − 0.5 × cost, which plan has the highest fitness?
2. Now use F = nutrition − 1.0 × cost. Which plan has the highest fitness?
3. With fitness-proportionate selection (the F from question 1), what is P(select C)? What goes wrong?
4. Which plan would you never pick, no matter how you weigh nutrition against cost? Why?

# Genetic Algorithms in Test Suite Generation

EvoSuite

# Genetic Algorithms for Test Suite Generation

To adapt GA to any task we need to define the following:

1. What is a chromosome / individual in the population?
   a. A test case!
2. How is fitness evaluated?
   a. Coverage!
3. How are individuals selected for breeding?
   a. “Rank Selection”
4. What happens during crossover?
   a. Take part of test1 and part of test2
5. What happens during mutation?
   a. Remove a statement, swap out a call, replace a primitive, etc....

# Initial Population

How do we generate a random initial population of test cases?

- Select a random length n
- Randomly insert n statements
- Many are invalid!
- Many are redundant

We need to do this in a more principled way... Java has rules

How did Randoop do this?

# Initial Population

When inserting random statement either:

1. Insert a call to the unit under test
   a. constructor or method
   b. Parameters: re-use existing values in the test or create a new variable

2. Insert a statement which can later be used as a parameter to a call to the unit under test

# Fitness

Fitness = Number of branches covered / total branches

Are some of these conflicting?

**We will discuss how to properly balance multiple objectives in GA later**

# Rank Selection

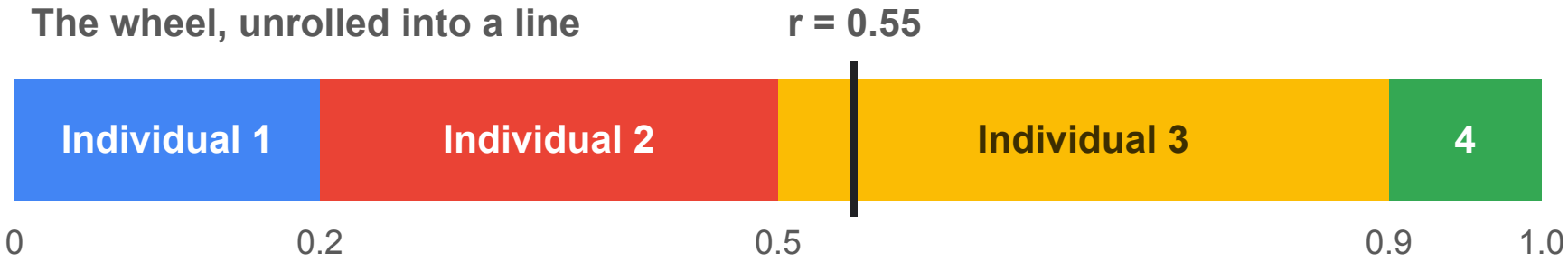
A method for choosing individuals to be parents for the next generation where

1. the population is first ranked based on their fitness, and then
2. selection probabilities are assigned based on their rank

This means the best individuals (with the highest rank) have a higher chance of being selected, while the worst individuals have a lower chance, regardless of the exact difference in their fitness values

How is this different from roulette? Pros / Cons?

# Roulette Wheel Selection




- Slice size = fitness / total fitness
- Running totals (cumulative): 0.2, 0.5, 0.9, 1.0, the right edge of each slice
- Spin: pick a random r between 0 and 1
- r = 0.55 lands between 0.5 and 0.9, so we pick Individual 3

# Rank Selection

1. Calculate fitness
2. Rank the population
    a. Sort the individuals based on their fitness values, assigning a rank to each individual (e.g., 1 for the best, N for the worst, where N is the population size)
3. Assign selection probabilities
    a. Based on the assigned ranks, determine the selection probability for each individual.
    - **Linear ranking:** A simple approach where the selection probability is directly proportional to the rank (best individual has the highest probability).

4. Select Parents
    a. Randomly select individuals for reproduction based on their assigned selection probabilities.

**Evosuite avoids scaling and cumulative probability by approximating**

# Rank Selection vs. Roulette

**Roulette: slice size = fitness / total**




**Rank: sort best to worst, weights 4, 3, 2, 1 (out of 10)**




- Fitness of individuals 1–4: 2, 3, 14, 1
- Roulette: one super-fit individual takes most of the line
- Rank: only the order matters, so the best is capped at 40%

# Exercise: Rank the Warmup

Warmup fitness (w = 0.5): A = 7.5, B = 8, C = −0.5, D = 6.5

Use linear rank selection with weights 4, 3, 2, 1 (out of 10)

What is each plan's selection probability?

Does anything break this time?

# Crossover

How do we create an offspring of two test cases?

1. Generate a random number between 0 and 1. This is the % of statements from each test to keep. If test1 has n1 statements and test2 has n2 statements
   a. `k = floor(r * n1) , j = floor(r * n2)`

2. New test1 = the first k statements from test1 and the last k statements from test2.
3. New test2 = the first j statements from test2 and the last j statements from test1.

4. Clean up test by removing unused variables

# Crossover

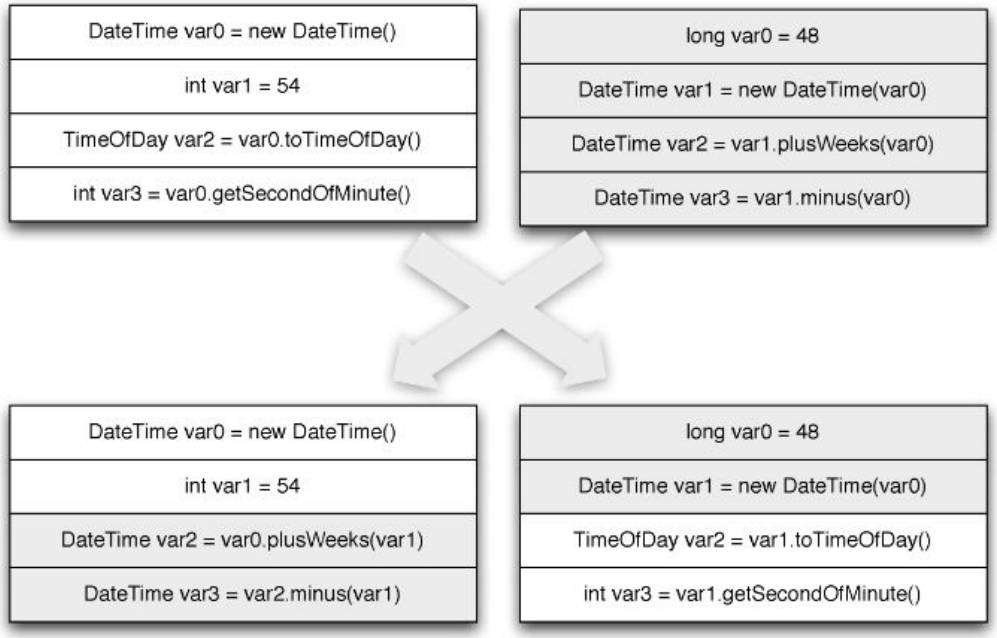



Fig. 4. Crossover between two test cases.

# Exercise: Make the Offspring

Test 1 has statements s1, s2, s3, s4

Test 2 has statements t1, t2, t3, t4, t5, t6

The random number is r = 0.5

What are k and j?

What are the two offspring?

# Mutation

1. Delete a statement
2. Insert a method call
3. Modify an existing statement
    a. Change callee (target object)
    b. Change params
    c. Change method / constructor - replace with one of the same return type
    d. Change field
    e. Change primitive

How do you think these program transformations are implemented?

# Exercise: Name That Mutation

Original test:

Account a = new Account(5); a.deposit(10);

Which mutation produced each version?

1. Account a = new Account(5);

2. Account a = new Account(5); a.deposit(-3);

3. Account a = new Account(5); a.deposit(10); a.withdraw(2);

# Assertions

Test suites contain a prefix and assertion

The GA we defined only generates a prefix. How does it find the assertion?

EvoSuite generates ***regression*** tests.

# Multiple Objective Optimization

# Genetic Algorithms with Multiple Objectives

Often times, our fitness function includes multiple objectives which may be conflicting.

- Meal Plan: Maximize nutritional value, minimize cost
- Employee Scheduling: Maximize coverage, minimize overtime
- Test Suite Generation: Maximize branch coverage

```
String example(int a) {
  switch (a) {
     case  0 : return "0";
     case  1 : return "1";
     case -1 : return "-1";
  default: return "default";
}
```

# Genetic Algorithms with Multiple Objectives

Fitness functions for multiple objectives:

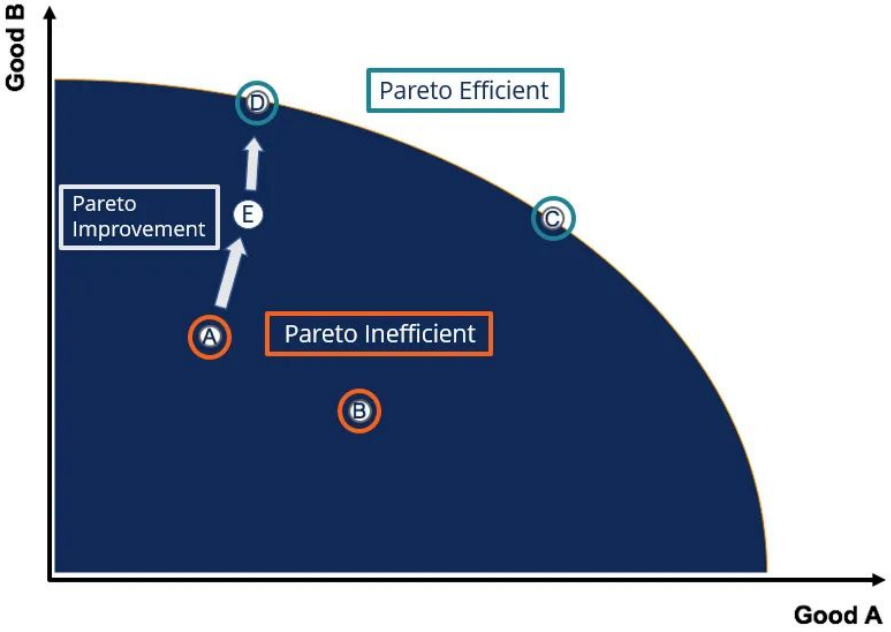
1. Weighted linear combination
    a. Nutritional Value + .5 * cost
    b. This becomes less effective with many goals
    c. Programs often have 100s of branches...

# Pareto Fronts

A **Pareto improvement** formalizes the idea of an outcome being "better in every possible way".

A change is called a **Pareto improvement** if it leaves at least one person in society better-off without leaving anyone else worse off than they were before.

A situation is called **Pareto efficient** or **Pareto optimal** if all possible Pareto improvements have already been made; in other words, there are no longer any ways left to make one person better-off, without making some other person worse-off



# Pareto Dominance

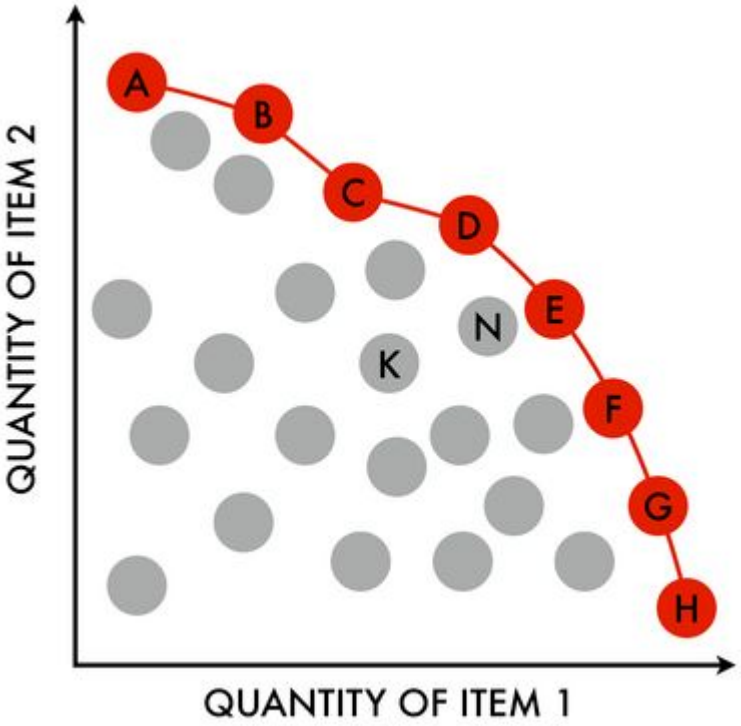
**Pareto Dominance**: A solution A dominates solution B if A is at least as good as B in all objectives and strictly better in at least one.

**Pareto Front**: The set of non-dominated solutions representing the trade-offs between conflicting objectives.

# Production Possibility Frontier

The red line is an example of a Pareto-efficient frontier, where the frontier and the area left and below it are a continuous set of choices.

The red points on the frontier are examples of Pareto-optimal choices of production. Points off the frontier, such as N and K, are not Pareto-efficient, **since there exist points on the frontier which Pareto-dominate them.**

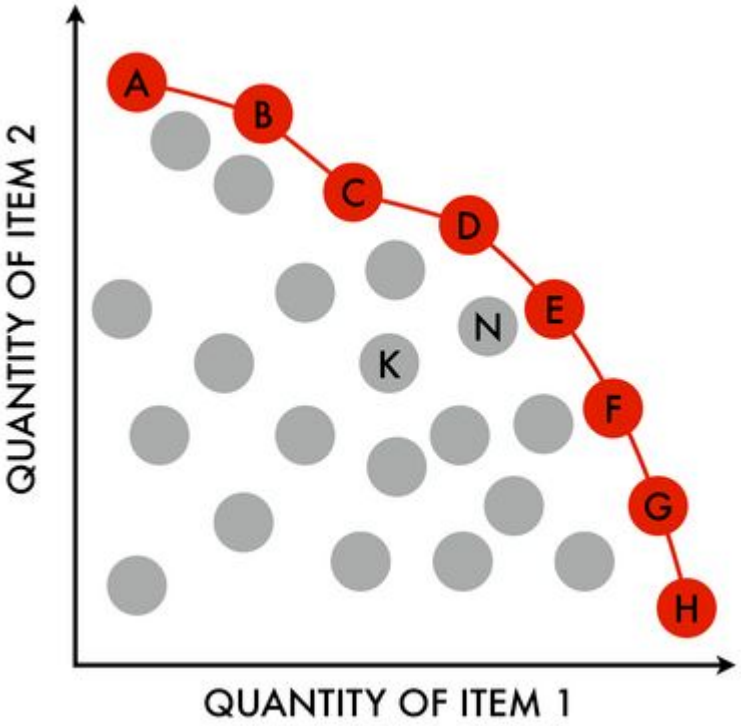


# Production Possibility Frontier

Points off the frontier, such as N and K, are not Pareto-efficient, **since there exist points on the frontier which Pareto-dominate them.**

a solution A dominates solution B if A is at least as good as B in all objectives and strictly better in at least one.

**What points dominate K and N?**



# Exercise: Find the Pareto Front

Each test covers the branches listed. Which tests are dominated? Which tests are on the Pareto front?

| Test | Branches covered |
|---|---|
| **T1** | B1T |
| **T2** | B1T, B2T |
| **T3** | B1F |
| **T4** | B1F, B2F |

# Back to EvoSuite

EvoSuite uses these concepts in its **selection function**

Calculates pareto optimal solutions

a. Tests which are not “dominated”
b. Dominance is when one test has better fitness on some branch and the other is not better on any other branch
c. If T1 coverages branch 4F, but T2 covers branch 4T, and are the same on all other branches, neither domains the other

# Branch Coverage Dominance Example

```
public void unlock(String pin) {
        //check if account holder is admin
        if (pin.equals(decode(admin))) { //B1
                locked = false;
                return;
        }
}
public boolean withdraw(double amount) {
        if (locked) return false; //B2

        if (amount > 0 && balance >= amount) { //B3
                balance -= amount;
                return true;
        }
        return false;
}
```

Dominance is when one test has better fitness on some branch and the other is not better on any other branch

| Test Case | Branches Covered |
|---|---|
| T1 | B2 T |
| T2 | B1 F, B2 T |
| T3 | B1 T, B2 F, B3 F |

T2 dominates T1

T3 and T2 do not dominate eachother

Making progress on B3 would make progress on B2 F worse

# EvoSuite Selection and Fitness

Last class, we simplified fitness: `total branches covered / total branches`

It's actually sorted by dominance ! All tests on the pareto front are equal

Higher branch coverage doesn't mean the test is necessarily better! We want our test suite to include both cases:

- Call to withdraw with locked
- Call to withdraw with unlocked

# DynaMosa

We want both T2 and T3 in our test suite, but if we keep B1F as a goal, we will never make progress toward covering B3T

DynaMosa dynamically adapts which goals are included in our ranking

Add T2 to the test suite and remove B1F from our goals

# DynaMosa

1. Generate initial population

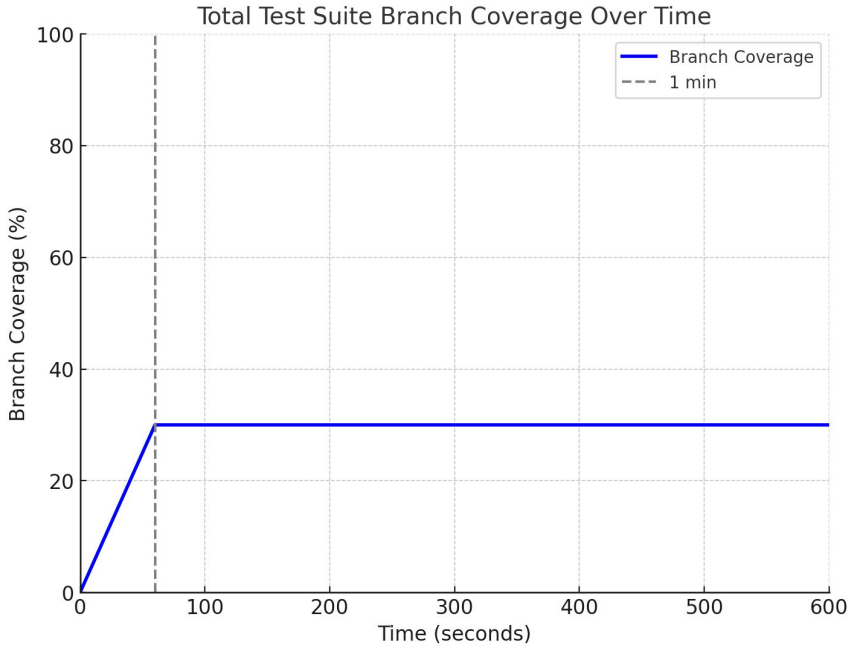
2. Until timeout:
   a. Select Parents
      i. **Requires evaluating population and ranking**
      ii. Rank based selection
   b. Perform Crossover
   c. Perform mutation with some probability
   **d. Dynamically adapt goals**

3. Output test suite

# CodaMosa: Escaping Coverage Plateaus with LLMs

# Coverage Plateaus

- SBST works well when mutation / crossover has a non-negligible likelihood of increasing fitness
- Imagine our unlock method...
  - What are the odds we would call unlock with the “magic pin” ?
- Fitness plateaus are common problem in test generation



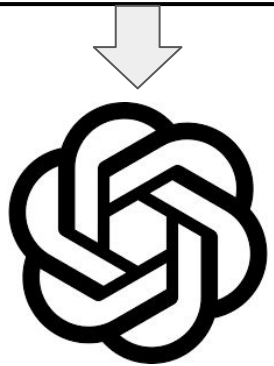
# CodaMosa

CodaMOSA, starts SBST and monitors its coverage progress.

When it notices a stall in coverage, it identifies methods that have low coverage

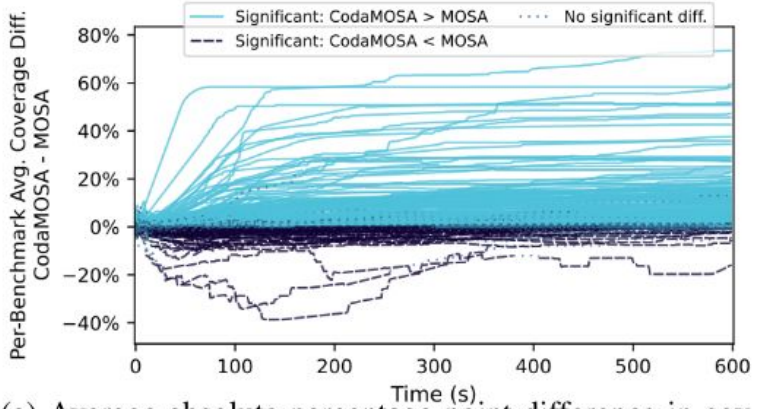
Then, it queries Codex to generate tests for these methods and adds them to the population

```
public void unlock(String pin) {
        //check if account holder is admin
        if (pin.equals(decode(admin))) { //B1
                locked = false;
                return;
        }
}
```

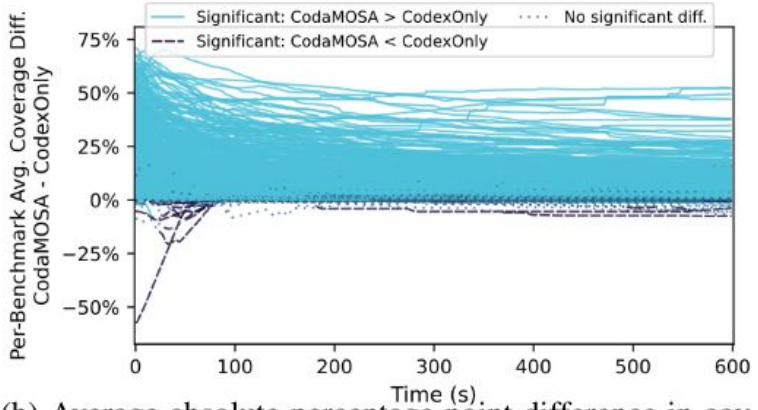


**unlock(“admin”)**

# CodaMosa Evaluation




(a) Average absolute percentage point difference in coverage between CODAMOSA and MOSA.




(b) Average absolute percentage point difference in coverage between CODAMOSA and CodexOnly.

# Summary

- EvoSuite adapts a GA to test generation: a chromosome is a test, fitness is coverage
- We traced a GA on meal plan
- Rank selection uses order, so negative fitness and outliers don’t break it
- Coverage has many objectives: EvoSuite keeps the Pareto front, and DynaMOSA drops goals once they’re covered